Problem Statement

Input:

1. A Scenic program P (which defines a distribution μ over scenes).
2. A view function $f\colon \text{Scene} \to Y$, which is a deterministic function over scenes. $|Y| < \infty$.
3. A *desired* distribution of $f(s)$, named ν, specified by the programmer through direct manipulation.

(1) and (2) together define the "pushforward" $f_{\#}\mu$: if $s \sim \mu$, then $f(s) \sim f_{\#}\mu$. $f_{\#}\mu$ is the distribution of $f(s)$. ν is what the programmer wants $f_{\#}\mu$ to be.

Output: An edited Scenic program P', which defines a distribution μ' over scenes such that $\nu = f_{\#}\mu'$.

Desired output properties:

1. μ' is picked such that $\mu(\cdot \mid f(s) = y) = \mu'(\cdot \mid f(s) = y)$ (i.e. the distribution of scenes in each bin y stays the same).
2. μ' is "similar" to μ on common measures of distance between probability distributions (minimizes the distance or gets close to it).
3. Sampling from P' is efficient.
4. P' is "similar" to P.

Jeffrey's Update

Let $w(s)$ be the Jeffrey weight of a scene s.

$$w(s) = \frac{\nu(f(s))}{(f_{\#}\mu)(f(s))} = \frac{\text{desired prob. of } f(s)}{\text{current prob. of } f(s)}$$

Since the Jeffrey weight only depends on the view value $f(s) = y$, we can also say that $w(y)$ is the Jeffrey weight of a view value y.

$$w(y) = \frac{\nu(y)}{(f_{\#}\mu)(y)} = \frac{\text{desired prob. of } y}{\text{current prob. of } y}$$

Jeffrey's update then says

$$\mu'(s) = \underbrace{w(s) \cdot \mu(s)}_{\substack{\text{scale each} \\ \text{scene probability} \\ \text{by } w(s)}} = \frac{\nu(f(s))}{(f_{\#}\mu)(f(s))}\,\mu(s) = \underbrace{\sum_y \nu(y)\,\mu(s \mid f(s) = y)}_{\substack{\text{law of total probability,} \\ \text{but with partition probabilities} \\ \text{set to } \nu(y)}}$$

where, for each y with $(f_{\#}\mu)(y) > 0$,

$$\mu(s \mid f(s) = y) = \frac{\mu(s) \cdot \mathbb{1}_{f(s)=y}}{(f_{\#}\mu)(y)}, \qquad \mathbb{1}_{f(s)=y} = \begin{cases} 1, & f(s) = y \\ 0, & \text{otherwise} \end{cases}$$

Note that Jeffrey's update is only defined when $(f_{\#}\mu)(y) = 0 \implies \nu(y) = 0$ (or equivalently $\text{supp}(\nu) \subseteq \text{supp}(f_{\#}\mu)$ in the discrete case). In other words, ν can't assign a non-zero probability to view values with 0 probability.

Program Patch

In general, we cannot compute $f_{\#}\mu$ exactly, so we estimate it from samples. Let $s_1, \ldots, s_n$ be drawn i.i.d. from μ, and let $\hat{\mu}$ be the estimate for μ, where

$$\hat{\mu}(s) = \frac{1}{n} \sum_{i=1}^{n} \mathbb{1}_{s_i = s}$$

$f_{\#}\hat{\mu}$ is our estimate of $f_{\#}\mu$.

$$(f_{\#}\hat{\mu})(y) = \frac{1}{n} \sum_{i=1}^{n} \mathbb{1}_{f(s_i) = y}$$

Let $\hat{Y} = \{y \in Y \mid (f_{\#}\hat{\mu})(y) > 0\}$ be the set of view values that appear in the samples. The programmer can only modify the probabilities of view values in $\hat{Y}$ so ν is specified on $\hat{Y}$, with $\sum_{y \in \hat{Y}} \nu(y) = 1$.

Our estimate for the Jeffrey weight for a particular view value y is then

$$\hat{w}(y) = \begin{cases} \dfrac{\nu(y)}{(f_{\#}\hat{\mu})(y)}, & y \in \hat{Y} \\ 1, & y \notin \hat{Y} \end{cases}$$

A rare view value can have $(f_{\#}\mu)(y) > 0$ and still be missing from the samples, so that $y \notin \hat{Y}$. To "leave it alone", we give it weight 1.

Define M to be the maximum Jeffrey weight for the set of scenes: $M = \max_{y \in Y} \hat{w}(y)$.

Define $\hat{a}(y)$ to be the acceptance probability of view value y, defined as $\hat{a}(y) = \frac{\hat{w}(y)}{M}$.

The program patch then adds require statement(s) for each possible $f(s) \in Y$. Let these be $y_1, \ldots, y_k$. Then, for each y_i, add

```
require not(f(s) == y_i) or u < a_hat(y_i)
```

where `u = Range(0, 1)` is a fresh uniform random value, so `u < a` holds with probability a.

Leaving out the require for y is not the same: it accepts y with probability 1, which is weight M rather than 1.

Properties of choosing $\hat{w}(y) = 1$ when $y \notin \hat{Y}$:

1. **GetPut holds.** If the programmer does not edit the histogram ($\nu = f_{\#}\hat{\mu}$ on $\hat{Y}$), every weight is 1, so $M = 1$, every require is always true, and P' defines the same distribution as P. Forbidding unseen view values instead ($\hat{w}(y) = 0$) would delete their scenes even though the programmer changed nothing.

2. **Rare scenes (mostly) stay at their original probability.** Let $m = \sum_{y \notin \hat{Y}} (f_{\#}\mu)(y)$ be the probability of the view values "missing" from the sample. $f_{\#}\hat{\mu}$ sums to 1 over $\hat{Y}$, but the true probability of $\hat{Y}$ is only $1 - m$, so $(f_{\#}\hat{\mu})(y) \approx \frac{(f_{\#}\mu)(y)}{1-m}$ for $y \in \hat{Y}$. The patch is then approximately Jeffrey's update to the extended target

$$\tilde{\nu}(y) = \begin{cases} (1-m)\,\nu(y), & y \in \hat{Y} \\ (f_{\#}\mu)(y), & y \notin \hat{Y} \end{cases}$$

which keeps the programmer's histogram on $\hat{Y}$ (scaled down to make room) and leaves the unseen view values at their original probability.

3. M **does not change.** Since $\sum_{y \in \hat{Y}} (f_{\#}\hat{\mu})(y) = 1$ and $\sum_{y \in \hat{Y}} (f_{\#}\hat{\mu})(y)\,\hat{w}(y) = \sum_{y \in \hat{Y}} \nu(y) = 1$, the weights on $\hat{Y}$ average to 1 under $f_{\#}\hat{\mu}$, so $\max_{y \in \hat{Y}} \hat{w}(y) \geq 1$. So $M = \max_{y \in \hat{Y}} \hat{w}(y)$: the unseen view values never set M. The Effective Sample Size section is also unaffected, since every cached sample has a view value in $\hat{Y}$.

Cost of Program Patch

Adding the require statements in the program patch increases the expected number of rejections. Under the patched program, scenes with $f(s) = y_i$ and $u \geq \hat{a}(y_i)$ will be rejected.

The probability that a scene passes the original requirements but is rejected by the new requirements is

$$q = \sum_{i=1}^{k} \underbrace{(f_{\#}\mu)(y_i)}_{\substack{\text{Probability of} \\ \text{view value } y_i}} \cdot \underbrace{(1 - \hat{a}(y_i))}_{\substack{\text{Probability that} \\ y_i \text{ gets rejected}}}$$

Let p_0 be the probability that a sample passes the original program P's requirements. Then, the patched program accepts with probability $p_0(1-q)$.

Let R_P and $R_{P'}$ be the number of rejections before a scene is accepted in the original program P and patched program P' respectively. Then, $R_P \sim \text{Geometric}(p_0)$ and $R_{P'} \sim \text{Geometric}(p_0(1-q))$.

The number of additional rejections due to the program patch Δ is then

$$\Delta = \mathbb{E}[R_{P'}] - \mathbb{E}[R_P] = \left(\frac{1}{p_0(1-q)} - 1\right) - \left(\frac{1}{p_0} - 1\right)$$

By using $f_{\#}\hat{\mu}$ as an estimator for $f_{\#}\mu$, we can work out that $q = 1 - \frac{1}{M}$. The view values $y \notin \hat{Y}$ have $(f_{\#}\hat{\mu})(y) = 0$, so they drop out of the sum:

$$\sum_{y \in Y} (f_{\#}\hat{\mu})(y) \cdot (1 - \hat{a}(y)) = \sum_{y \in \hat{Y}} (f_{\#}\hat{\mu})(y) \cdot \left(1 - \frac{\nu(y)}{M(f_{\#}\hat{\mu})(y)}\right) = \sum_{y \in \hat{Y}} (f_{\#}\hat{\mu})(y) - \sum_{y \in \hat{Y}} \frac{\nu(y)}{M} = 1 - \frac{1}{M}$$

Using this, the number of additional rejections is $\Delta = \frac{M-1}{p_0}$. The expected number of attempts per accepted scene goes from $\frac{1}{p_0}$ to $\frac{M}{p_0}$; the attempts are scaled up by exactly M.

Effective Sample Size

Instead of sampling from the patched program, we can reuse the cached samples $s_1, \ldots, s_n \sim \mu$ by giving each sample its estimated Jeffrey weight $\hat{w}(f(s_j))$. Weighted averages over the cached samples then estimate averages under μ' (importance sampling). But weighted samples are worth less than fresh samples from μ': a few samples with large weights dominate every average. The *effective sample size* (ESS) measures this loss. It is approximately the number of i.i.d. samples from μ' that would give estimates with the same variance as the n weighted samples.

Normalize the weights so they sum to 1:

$$\bar{w}_j = \frac{\hat{w}(f(s_j))}{\sum_{l=1}^{n} \hat{w}(f(s_l))}$$

Kish's ESS is then

$$\text{ESS} = \frac{1}{\sum_{j=1}^{n} \bar{w}_j^2} = \frac{\left(\sum_{j=1}^{n} \hat{w}(f(s_j))\right)^2}{\sum_{j=1}^{n} \hat{w}(f(s_j))^2}$$

$1 \leq \text{ESS} \leq n$. $\text{ESS} = n$ when all weights are equal (i.e. $\nu = f_{\#}\hat{\mu}$, so the edit changes nothing), and $\text{ESS} = 1$ when one sample has all the weight.

Since $\hat{w}$ only depends on the view value, we can group the samples by view value. Because $\hat{\mu}$ was estimated from these same samples, there are exactly $n \cdot (f_{\#}\hat{\mu})(y_i)$ samples with $f(s_j) = y_i$, each with weight $\frac{\nu(y_i)}{(f_{\#}\hat{\mu})(y_i)}$. Assuming $\sum_{i=1}^{k} \nu(y_i) = 1$ (i.e. ν puts no mass on unobserved view values),

$$\sum_{j=1}^{n} \hat{w}(f(s_j)) = \sum_{i=1}^{k} n(f_{\#}\hat{\mu})(y_i) \cdot \frac{\nu(y_i)}{(f_{\#}\hat{\mu})(y_i)} = n, \qquad \sum_{j=1}^{n} \hat{w}(f(s_j))^2 = n \sum_{i=1}^{k} \frac{\nu(y_i)^2}{(f_{\#}\hat{\mu})(y_i)}$$

so

$$\text{ESS} = \frac{n}{\sum_{i=1}^{k} \frac{\nu(y_i)^2}{(f_{\#}\hat{\mu})(y_i)}}$$

The ESS is small when ν puts a lot of mass on view values that are rare under $f_{\#}\hat{\mu}$.

Comparison with the program patch: Applying the patch's `require` statements to the cached samples would keep s_j with probability $\hat{a}(f(s_j))$, so on average it keeps $\sum_{i=1}^{k} n(f_{\#}\hat{\mu})(y_i) \cdot \hat{a}(y_i) = \frac{n}{M}$ samples. Since $\frac{\nu(y_i)}{(f_{\#}\hat{\mu})(y_i)} = \hat{w}(y_i) \leq M$,

$$\sum_{i=1}^{k} \frac{\nu(y_i)^2}{(f_{\#}\hat{\mu})(y_i)} \leq M \sum_{i=1}^{k} \nu(y_i) = M \quad \Longrightarrow \quad \text{ESS} \geq \frac{n}{M}$$

So, measured by ESS, reweighting the cached samples always keeps at least as much of the cache as rejecting them.

Caveats:

1. The ESS only looks at the weights. If $\nu(y) > 0$ for a view value y with $(f_{\#}\hat{\mu})(y) = 0$, no cached sample can represent y, and the ESS will not warn about it.
2. The ESS is a rule of thumb. It does not depend on which quantity is being estimated, so the actual variance of a particular estimate can be better or worse.